0131. Palindrome Partitioning

Medium · Backtracking

leetcode.com/problems/palindrome-partitioning

Problem

- Split s into pieces where every piece is a palindrome; return all such partitions.
- Example: "aab" → [["a","a","b"],["aa","b"]].

Intuition

- Choose the first piece s[start..end] (only if it is a palindrome), then partition the rest recursively.
- Precomputing isPal[i][j] with DP avoids rechecking substrings.

Brute Force

- Try all 2^(n-1) cut patterns and validate each piece.

Optimal Approach

- IsPal[i][j] = s[i]==s[j] && (j - i < 2 || isPal[i+1][j-1]), filled from the bottom up.
- Dfs(start): if start == n record; for end in start..n-1 with isPal[start][end]: add, dfs(end+1), remove.

Worked Example

```
"aab".

"a" | rest "ab": "a" | "b"      -> [a,a,b]
"aa"| rest "b" : "b"            -> [aa,b]
"aab" not a palindrome -> skip
```

Java Solution

```java
public class PalindromePartitioning {
    public List<List<String>> partition(String s) {
        int n = s.length();
        boolean[][] pal = new boolean[n][n];
        for (int i = n - 1; i >= 0; i--)
            for (int j = i; j < n; j++)
                pal[i][j] = s.charAt(i) == s.charAt(j) && (j - i < 2 || pal[i + 1][j - 1]);
        List<List<String>> res = new ArrayList<>();
        dfs(s, 0, pal, new ArrayList<>(), res);
        return res;
    }

    private void dfs(String s, int start, boolean[][] pal, List<String> path, List<List<String>> res) {
        if (start == s.length()) { res.add(new ArrayList<>(path)); return; }
        for (int end = start; end < s.length(); end++) {
            if (!pal[start][end]) continue;
            path.add(s.substring(start, end + 1));
            dfs(s, end + 1, pal, path, res);
            path.remove(path.size() - 1);
        }
    }
}
```

Complexity

- Time: O(n 2^n). Space: O(n^2) for the table.

Pitfalls

- Fill isPal with i descending so isPal[i+1][j-1] is ready.